

Load Balancer para Modelos de IA

Artur Fernandes Braga de Menezes¹, Artur Rizzi Martinho¹,
Felipe Lacerda Tertuliano¹, Gabriel Alves da Silva Diógenes¹,
Rafael Mortimer Colares¹

¹Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
30.535-901 - Belo Horizonte - MG - Brazil

Abstract. *Ullamco nostrud aliquip proident ex ullamco ullamco nostrud adipiscing. Non commodo incididunt veniam pariatur veniam sit. Duis consequat esse enim deserunt dolor exercitation. Quis consequat velit deserunt sint. Exercitation deserunt labore veniam quis ad enim quis commodo culpa incididunt aute veniam voluptate. Dolor aliqua aliqua non aliqua. In ex aute eiusmod voluptate exercitation proident eu minim magna aliqua fugiat mollit et veniam.*

Resumo. *Ullamco nostrud aliquip proident ex ullamco ullamco nostrud adipiscing. Non commodo incididunt veniam pariatur veniam sit. Duis consequat esse enim deserunt dolor exercitation. Quis consequat velit deserunt sint. Exercitation deserunt labore veniam quis ad enim quis commodo culpa incididunt aute veniam voluptate. Dolor aliqua aliqua non aliqua. In ex aute eiusmod voluptate exercitation proident eu minim magna aliqua fugiat mollit et veniam.*

1. Introdução

A evolução dos modelos de Inteligência Artificial (IA), especialmente dos Grandes Modelos de Linguagem (Large Language Models — LLMs), tem ampliado significativamente a capacidade dos sistemas computacionais de realizar tarefas como geração e compreensão de texto, programação e processamento de informações. Entretanto, o aumento da capacidade desses modelos também está associado a uma maior demanda por recursos computacionais durante sua utilização. Além disso, diferentes aplicações podem apresentar requisitos distintos de processamento, latência e utilização de recursos, tornando o gerenciamento eficiente das requisições um desafio relevante para sistemas que disponibilizam modelos de IA em larga escala. Estudos sobre sistemas de serving de LLMs destacam justamente a heterogeneidade das requisições e a necessidade de mecanismos eficientes para distribuir e agendar sua execução [Kossmann et al. 2024, Sun et al. 2024].

Nesse contexto, utilizar um único modelo de grande porte para atender diferentes tipos de requisições pode resultar em uma utilização pouco eficiente dos recursos disponíveis, principalmente quando determinadas tarefas poderiam ser executadas por modelos menores ou especializados. O problema também se estende à distribuição das requisições entre diferentes instâncias, uma vez que variações na demanda podem provocar sobrecarga em determinados recursos enquanto outros permanecem subutilizados. Trabalhos recentes demonstram que estratégias de balanceamento e escalonamento podem melhorar a utilização da infraestrutura disponível. O SkyWalker, por exemplo, propõe um mecanismo de balanceamento de carga entre diferentes regiões para inferência de LLMs, buscando utilizar os recursos de forma mais eficiente e reduzir custos de serving [Xia et al. 2025]. De forma semelhante, o Llumnix explora o reagendamento dinâmico

de requisições entre múltiplas instâncias de modelos para lidar com diferentes requisitos de recursos e latência [Sun et al. 2024].

Diante desse cenário, este trabalho propõe o estudo e desenvolvimento de um Load Balancer voltado para modelos de IA, capaz de distribuir as requisições entre diferentes modelos de acordo com suas características e com a disponibilidade dos recursos. A utilização de múltiplos modelos pode permitir que diferentes tipos de solicitações sejam direcionados para alternativas mais adequadas, evitando o uso desnecessário de modelos de maior porte quando modelos mais simples forem suficientes para determinada tarefa. Dessa forma, busca-se investigar os possíveis impactos dessa abordagem sobre aspectos como utilização dos recursos computacionais, tempo de resposta e custo de processamento. A literatura demonstra que mecanismos de balanceamento e escalonamento aplicados ao serving de LLMs podem proporcionar ganhos de desempenho e redução de custos, reforçando a relevância de investigar estratégias semelhantes em arquiteturas compostas por múltiplos modelos [Kossmann et al. 2024, Xia et al. 2025, Sun et al. 2024].

2. Revisão bibliográfica

Revisão bibliográfica aqui

3. Metodologia de Desenvolvimento e Avaliação

A metodologia adotada neste trabalho organiza-se em torno de ciclos semanais de desenvolvimento e avaliação, conforme apresentado na Tabela 1. Se espera ter até a sexta semana um código completo capaz de atender nossos requisitos e ser utilizado para as medições, a partir disso o foco do trabalho se vira para coleta de resultados, melhorias, redação do artigo e apresentações.

Quanto ao método de avaliação e validação foi escolhida a medição em ambiente controlado complementada pela análise dos resultados, isso porque o desempenho de um LoadBalancer para modelos de IA depende de fatores dinâmicos do ambiente que não podem ser capturados por simulações. A validação será conduzida por meio de experimentos comparativos entre dois cenários: (i) *baseline*, no qual todas as requisições são direcionadas ao modelo de maior porte; e (ii) *proposta*, na qual o LoadBalancer classifica o prompt e o encaminha ao modelo mais adequado. Serão coletadas métricas de latência média, P95 e P99, além de uso de RAM e GPU por máquina, a análise estatística dos dados incluirá identificação de outliers e geração de gráficos comparativos.

O ambiente experimental será composto por máquinas executando o Ollama com modelos leves (Phi-3.5-mini, Qwen2.5-Coder-0.5B e AILO-152M-v2), e o LoadBalancer será implementado em Python com FastAPI, utilizando `httpx` para comunicação com os servidores de inferência. Ajustes podem envolver a inclusão de novas métricas, a alteração do conjunto de modelos avaliados ou a adoção complementar de técnicas de simulação, caso se mostrem necessárias para validar aspectos específicos do sistema.

4. Resultados

Consectetur quis cillum eu minim Lorem minim anim dolor et. Ut incididunt est enim Lorem commodo occaecat sunt nulla adipisicing fugiat. Quis amet nisi do deserunt laborum. Labore velit deserunt excepteur et velit adipisicing consectetur ipsum. Enim dolore eiusmod proident exercitation ullamco commodo veniam anim ad ut ullamco Lorem Lorem.

Tabela 1. Cronograma de atividades do projeto.

Semana	Atividades principais
1	Configuração do ambiente (Ollama, modelos leves), levantamento bibliográfico inicial.
2	Desenvolvimento do classificador de prompts, implementação da versão inicial do LoadBalancer.
3 (Sprint 2)	Detalhamento da implementação, coleta de resultados preliminares, preparação e apresentação oral do Sprint 2.
4	Refinamento do classificador, implementação de métricas de monitoramento, escrita da metodologia.
5	Expansão do conjunto de testes, execução de experimentos comparativos, análise estatística dos resultados.
6	Otimização do LoadBalancer, testes automatizados de roteamento, escrita da seção de resultados.
7	Comparação com trabalhos relacionados, redação da seção comparativa, preparação de material visual para o pitch.
8	Redação das conclusões, revisão e refinamento do artigo, roteiro do pitch comercial.
9 (Sprint 3)	Gravação e edição do pitch comercial, preparação e apresentação oral final.
10	Submissão final do artigo, documentação do projeto e entrega dos materiais de apresentação.

5. Conclusão

Nulla velit laboris ut in in ullamco minim id cillum magna quis ullamco in. Veniam proident proident reprehenderit nostrud aliqua incididunt adipisicing ullamco deserunt irure proident ea officia. Tempor elit minim voluptate adipisicing laborum.

Referências

- Kossmann, F., Fontaine, B., Khudia, D., Cafarella, M., and Madden, S. (2024). Is the gpu half-empty or half-full? practical scheduling techniques for llms. *arXiv preprint arXiv:2410.17840*.
- Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. (2024). Llum-nix: Dynamic scheduling for large language model serving. *arXiv preprint arXiv:2406.03243*.
- Xia, T., Mao, Z., Kerney, J., Jackson, E. J., Li, Z., Xing, J., Shenker, S., and Stoica, I. (2025). Skywalker: A locality-aware cross-region load balancer for llm inference. *arXiv preprint arXiv:2505.24095*.